Aarhus University

Computer Science

Numerical Linear Algebra

Handin 9

Author
Søren M. Damsgaard

Student number
202309814

April 26, 2026

Zulu Bravo PsyOp Striker 9

Given a system of flowing thingies 1 with the following information:

1. A contains 300 L of mixture and 84 g of salt.
2. B contains 100 L of muxture and 36 g of salt.
3. a flows with 30 $L/\min$ of water
4. b flows with 15 $L/\min$ of mixture

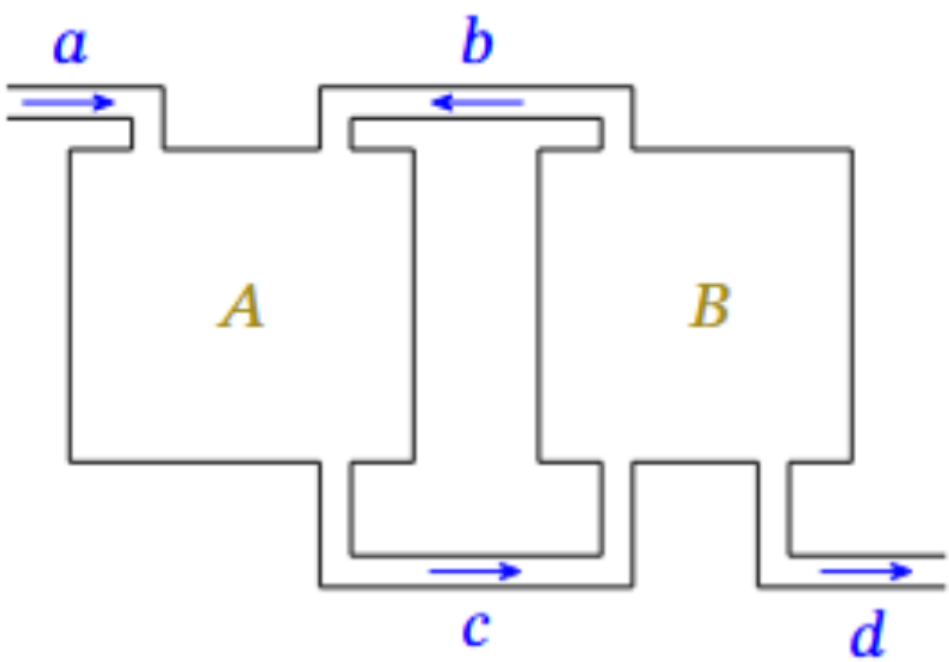

Figure 1: Containers connected by pipes: Flowchart of the system.

The Equalizer

Determining the flow of c and d so as the amount of liquid in both containers are constant.

1. A has an input a and b which totals to 45 $L/\min$, so c needs to flow with 45 $L/\min$ to cancel out the input from a and b.
2. B has an output b and a newly determined input c, so the following should determine the flow of the remaining pipe $c - b = 30\ L/\min = d$

Two Peens In A System

Explain why the salt amounts $y_0(t)$ in A and $y_1(t)$ in B satisfy the system

$$y_0'(t) = -0.15y_0(t) + 0.15y_1(t)$$
$$y_1'(t) = 0.15y_0(t) - 0.45y_1(t)$$

The change in salt corresponds to the input and output, percentages, of mixture in each container.
for A: c outputs 45 $L/\min$ which is 15% of A hence $-0.15y_0(t)$, and b inputs 15 $L/\min$ which is 15% of B hence $+0.15y_1(t)$.
for B: c inputs so it's $+0.15y_0(t)$ and b outputs 45 $L/\min$ which is 45% of B hence $-0.45y_1(t)$.

This give us a system that can be written as:

$$y'(t) = Ay = \begin{bmatrix} -0.15 & 0.15 \\ 0.15 & -0.45 \end{bmatrix} \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix}$$

with the initial value:

$$y(0) = \begin{bmatrix} 84 \\ 36 \end{bmatrix}$$

To Eigen or Not To Eigen

The Author calls upon god to protect me against the horros and impending doom of using a numpy function that isn't explicitely shown in the Bibel of Numerical Linear Algebra by Jesus AKA Andrew Swan.

```python
import numpy as np

A = np.array([[-0.15, 0.15],
              [0.15, -0.45]])
# Please don't end me... \(;_;)/
eigvalues, eigvectors = np.linalg.eig(A)

print('Eig Values:\n', eigvalues)
print('Eig Vectors:\n', eigvectors)

# Output (not part of the actual code, just for reference):
Eig Values:
 [-0.08786797 -0.51213203]
Eig Vectors:
 [[ 0.92387953 -0.38268343]
 [ 0.38268343  0.92387953]]
```

Reducing the Eig thingies to just two significant values.

$$\text{Eigenvalues} \qquad \lambda_0 = -0.09 \qquad \lambda_1 = -0.51 \tag{1}$$

$$\text{Eigenvectors} \qquad v_0 = \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} \qquad v_1 = \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} \tag{2}$$

Solutions for $y_0(t)$ and $y_1(t)$

Finding a solution for $y_0'(t)$ and $y_1'(t)$ using the eigen thingies, proposition 22.3 [1] says a solution is given using the eigenvalues and eigenvectors as such:

$$y(t) = c_0 v_0 e^{\lambda_0 t} + c_1 v_1 e^{\lambda_1 t}$$

Inserting the values from (1) and (2).

$$y(t) = c_0 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} e^{-0.09t} + c_1 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} e^{-0.51t} \tag{3}$$

To isolate and determine c_0 and c_1, the initial state for $y(0)$ can be used.

$$y(0) = \begin{bmatrix} 84 \\ 36 \end{bmatrix} = c_0 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} + c_1 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} = \begin{bmatrix} c_0 \cdot 0.92 + c_1 \cdot -0.38 \\ c_0 \cdot 0.38 + c_1 \cdot -0.92 \end{bmatrix}$$

This gives us the following system:

$$\begin{aligned} 0.92c_0 - 0.38c_1 &= 84 \\ 0.38c_0 + 0.92c_1 &= 36 \end{aligned}$$

Using the god-given gift of numpy:

```
B = np.array([[0.92, -0.38],
              [0.38, 0.92]])

b = np.array([84, 36])

c0, c1 = np.linalg.solve(A, b)

print('c0 = ', c0)
print('c1 = ', c1)

# Output for reference (not part of the actual code)
c0 = 91.80460234154218
c1 = 1.2111425111021366
```

Rounding down c_0 and c_1

$$c_0 = 91.80 \qquad c_1 = 1.21$$

Now it's possible to determine a solution for both $y_0(t)$ and $y_1(t)$, by using the newfound constants and plotting them into (3)

$$y(t) = 91.80 \begin{bmatrix} 0.92 \\ 0.38 \end{bmatrix} e^{-0.09t} + 1.21 \begin{bmatrix} -0.38 \\ 0.92 \end{bmatrix} e^{-0.51t} = \begin{bmatrix} 84.456e^{-0.09t} - 0.4598e^{-0.51t} \\ 34.884e^{-0.09t} + 1.1132e^{-0.51t} \end{bmatrix}$$

Plotting

Plotting both y_0 and y_1 versus time, shows an exponential decrease in salt over time which is expected since the biggest term -0.45 represents the output in container B from pipe d.

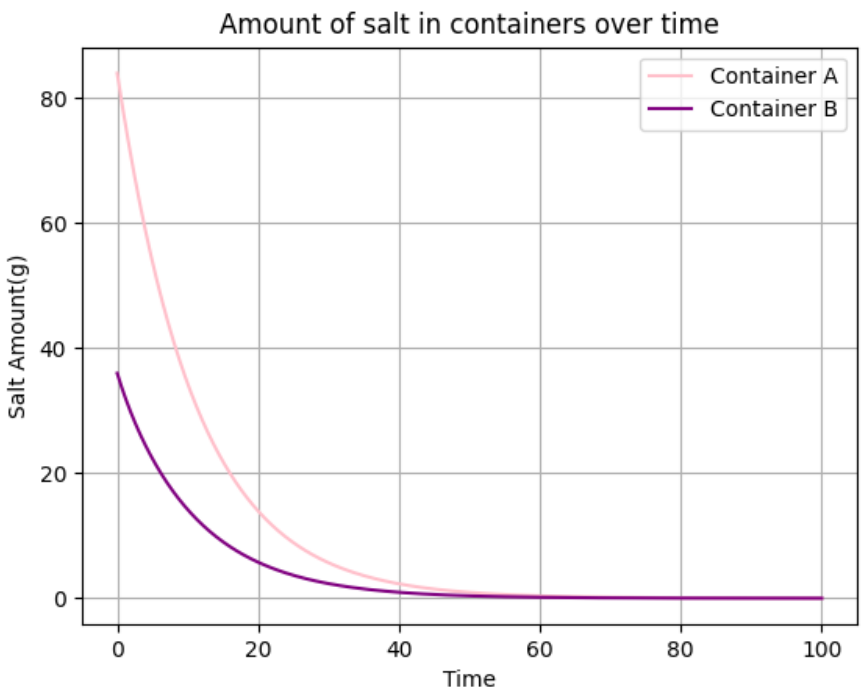

Figure 2

For the y_0 versus y_1 it shows the relation in how the containers salt levels are related.

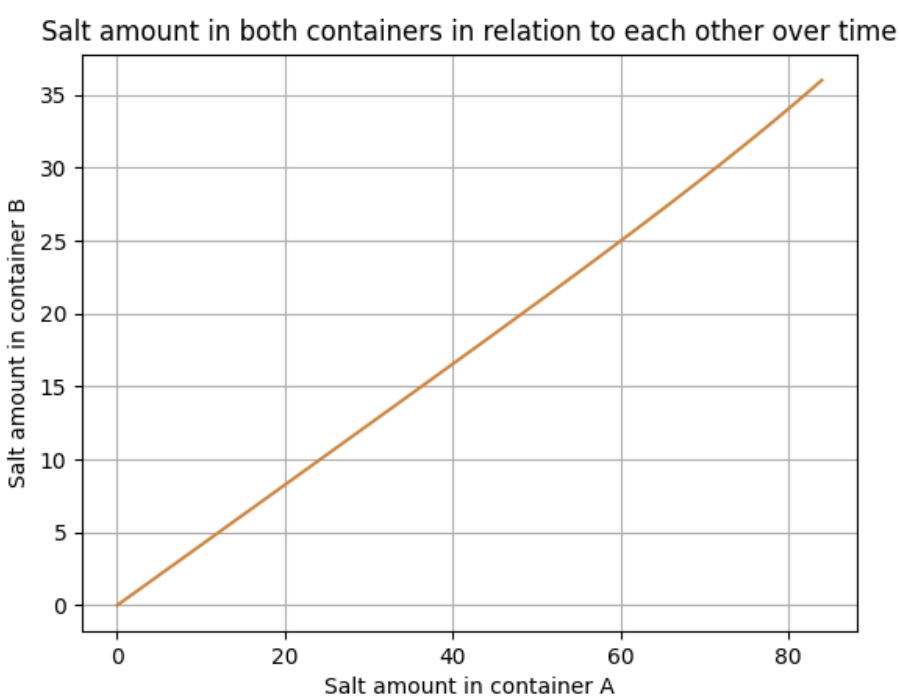

Figure 3

This specifically shows a linear relation in the salt levels, since both container A and B drain the same rate over time.

Limits of ∞

Taking (3) and splitting it into y_0 and y_1

$$y_0(t) = c_0 0.92 e^{-0.09t} - c_1 0.38 e^{-0.51t}$$
$$y_1(t) = c_0 0.38 e^{-0.09t} + c_1 0.92 e^{-0.51t}$$

it's know that $e^{-t} \to 0$ as $t \to \infty$ so the solution will approach zero as t grows, which makes sense in regards to 2 and 3.
It also means that the terms that decay the fastest can be seen as neglible, so any terms with $e^{-0.51t}$

$$\lim_{t\to\infty} \frac{y_1(t)}{y_0(t)} = \frac{c_0 0.38 e^{-0.09t}}{c_0 0.92 e^{-0.09t}} = \frac{0.38}{0.92} \approx 0.41 \tag{4}$$

Which is the ratio between the salt level in A and B as t changes.

Appendix

```python
import numpy as np
import matplotlib.pyplot as plt

# Finding Eigenvalues/vectors (c)
A = np.array([[-0.15, 0.15],
              [0.15, -0.45]])
# Please don't end me... \(;_;)/
eigvalues, eigvectors = np.linalg.eig(A)


# solving for the constants c_0 and c_1 in (d)
B = np.array([[0.92, -0.38],
              [0.38, 0.92]])

    b = np.array([84, 36])

c0, c1 = np.linalg.solve(B, b)


# (e)
t = np.linspace(0,100,1000)
y_0 = 84.456 * np.e**(-0.09*t) - 0.4598 * np.e**(-0.51*t)
y_1 = 34.884 * np.e**(-0.09*t) + 1.1132 * np.e**(-0.51*t)


## Brr brrr printing section
print(f'Eig Values:\n {eigvalues}')
print(f'Eig Vectors:\n {eigvectors}' )
print('c0 = ', c0)
```

```python
print('c1 = ', c1)

fig1, y = plt.subplots()
y.plot(t,y_0, color='pink', label='Container A')
y.plot(t, y_1, color='purple', label='Container B')
y.set_title('Amount of salt in containers over time')
y.set_xlabel('Time')
y.set_ylabel('Salt Amount(g)')
y.grid()
y.legend()

fig2, pl = plt.subplots()
pl.plot(y_0,y_1, color='peru')
pl.set_xlabel('Salt amount in container A')
pl.set_ylabel('Salt amount in container B')
pl.grid()
pl.set_title('Salt amount in both containers in relation to each other
    over time')

plt.show()
```

References

[1] Andrew Swan. *Numerisk Lineær Algebra*. Institut for Matematik, Aarhus Universitet, 2025.